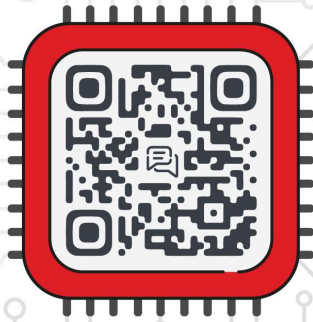




14

11

2100 sp

نظري

محتوى مجاني غير مخصص للبيع التجاري

كلية الهندسة المعلوماتية

السنة الرابعة

البرمجيات الوسيطة Middleware

د. عمار جوخدار



RB Informatix; 08/06/2026

برمجة تفرعية

في المحاضرة السابقة تم توضيح أن بناء البرامج بصورة صحيحة لا يعتمد فقط على كتابة خوارزميات تحقق النتائج المطلوبة، وإنما على القدرة على التعامل مع جميع حالات الإدخال الممكنة. فالبرنامج في جوهره يمثل علاقة بين الدخل (Input) والخرج (Output)، وتكمن صعوبة البرمجة في ضمان صحة هذه العلاقة لجميع الحالات وليس لبعضها فقط، إذ إن معظم الأخطاء البرمجية لا تنتج عن خطأ في الحل نفسه، وإنما عن عدم اكتماله وعدم تغطيته لجميع حالات الإدخال.

كما تمت مناقشة مفهوم المجموعات القابلة للعد بشكل عودي (Recursively Enumerable Sets)، والتي تمثل معظم التطبيقات البرمجية، حيث لا يمكن تعداد جميع عناصرها بشكل مباشر، وإنما يتم تعريفها باستخدام عدد محدود من القواعد الاستقرائية. وأكدنا أن العودية (Recursion) هي خاصية في بنية مجموعة الإدخال نفسها وليست خاصية في البرنامج؛ فالبرنامج لا يخترع طريقة الحل، وإنما يتبع الوصف العودي للمجموعة ويعالج عناصرها وفق هذه القواعد.

وانطلاقاً من ذلك، تم استعراض مفهوم التقطيع البنيوي (Structural Decomposition)، الذي يقوم على تفكيك الدخل إلى مكونات أبسط، بحيث يصبح حل المشكلة مبنياً على حل هذه المكونات ودمج نتائجها. وعند توصيف مجموعة الإدخال بهذه الطريقة، تصبح بنية البرامج متشابهة إلى حد كبير؛ إذ يكفي أن يتبع البرنامج القواعد التي بُنيت بها المجموعة، وهو المبدأ الذي تعتمد عليه تطبيقات عديدة مثل المترجمات (Compilers) التي تبني عملية التحليل اعتماداً على القواعد العودية للغة.

ويسهم هذا الأسلوب في كتابة برامج أكثر تنظيماً، وأسهل في الصيانة والتطوير والعمل الجماعي، لأنه يعتمد على قواعد عامة بدلاً من معالجة كل حالة بصورة منفصلة.

ويستثنى من ذلك تطبيقات الذكاء الاصطناعي، فهي لا تعتمد على قواعد منطقية تصف جميع الحالات كما في البرمجة التقليدية، وإنما تعتمد على التعلم من عدد كبير من الأمثلة، ثم تقارن المدخلات الجديدة بما تعلمته سابقاً للوصول إلى النتيجة الأكثر احتمالاً.

نمط التصميم Factory

ملاحظة أريسيزية :

أشار الدكتور إلى أنه لن يتوسع في شرح بقية أنماط التصميم (Design Patterns)، لأنها ستُدرس بشكل مفصل في مقرر مستقل، لذلك تجاوز عدداً من الأنماط مثل Composite و Strategy و Observer. إلا أنه توقف عند نمط المصنع (Factory Pattern) لارتباطه المباشر ببنية Facade والأنظمة الموزعة التي تتناولها هذه المادة.

فكرة Factory

يستخدم نمط Factory عندما توجد أكثر من طريقة لتنفيذ الطلب نفسه، أو عندما يكون هناك أكثر من خدمة أو أكثر من مكون قادر على معالجة الرسالة.

فعند وصول رسالة (Request) إلى النظام، يكون المطلوب أولاً تحديد:

1. ال Aspect الذي يجب تطبيقه على الرسالة.
2. ال Service المناسبة لتنفيذ الطلب.

وتتمثل مهمة ال Factory في مطابقة (Matching) محتوى الرسالة مع الجهة المناسبة لمعالجتها، ثم توجيه الطلب إليها.

العلاقة بين Facade و Factory:

- تُستخدم ال Factory غالباً داخل طبقة Facade.
- فال Facade تستقبل الطلبات القادمة من المستخدم، بينما تتولى ال Factory تحليل محتوى الرسالة واتخاذ قرار التوجيه، أي تحديد ال Aspect المناسب والخدمة (Service) التي يجب تنفيذها.
- وبذلك تصبح ال Facade مسؤولة عن استقبال الطلبات، في حين تكون ال Factory مسؤولة عن اختيار المسار الصحيح لتنفيذها.

تشبيه ساعي البريد

- يمكن تشبيه ال Factory بمساعد البريد.
- فعندما تصل رسالة إلى مركز البريد، لا يكون المطلوب من موظف الاستقبال معرفة صاحب الرسالة، وإنما يقوم مساعد البريد بقراءة العنوان الموجود عليها وتحديد المنزل الذي يجب أن تُرسل إليه.
- وبالمثل، تستقبل ال Facade الرسالة، بينما تقوم ال Factory بقراءة المعلومات الموجودة فيها وتحديد الجهة المناسبة التي يجب أن تستلم الطلب.

دور Factory في الأنظمة الموزعة:

لا يقتصر دور الـ Factory على اختيار الخدمة المناسبة، وإنما يمتد أيضاً إلى إدارة الموارد في الأنظمة الموزعة، حيث يمكنها اختيار المخدم (Server) الأنسب لتنفيذ الطلب. وتحقق هذه الآلية عدداً من الفوائد، أهمها:

1. توزيع الأحمال (Load Balancing) بين المخدمات.
2. الاستفادة من الموارد المتاحة.
3. زيادة تحمل الأعطال (Fault Tolerance)، بحيث يستطيع النظام متابعة العمل حتى عند توقف أحد المخدمات.

استخدام Factory مع طبقة DAO

- تُستخدم الـ Factory أيضاً داخل طبقة DAO (Data Access Object) عند توزيع البيانات على أكثر من قاعدة بيانات.
- فبدلاً من أن يعرف البرنامج مكان تخزين كل كائن، تتولى الـ Factory تحديد قاعدة البيانات الفيزيائية (Physical Database) المناسبة، سواء كانت موجودة على Oracle أو SQL Server أو أي مخدم آخر، ثم يتم تنفيذ عملية القراءة أو الكتابة عليها دون أن يتعامل البرنامج مع تفاصيل مكان التخزين.

أمثلة:

تم ذكر عدة أمثلة لتوضيح فكرة الـ Factory، منها:

1. اختيار الخدمة (Service Selection): عند طلب خدمة مثل أسعار العملات، تحدد الـ Factory الخدمة المسؤولة عن تنفيذ الطلب.
2. فيسبوك: عند طلب منشور أو صورة، تكون البيانات موزعة على عدة مخدّمات، فتحدد الـ Factory المخدم الذي يحتوي على البيانات المطلوبة أو الأنسب للاستجابة.
3. طبقة DAO: عند طلب كائن معين، تحدد الـ Factory قاعدة البيانات الفيزيائية التي يوجد فيها هذا الكائن ثم تنفذ العملية عليها.

ملاحظة:

ركز الدكتور على الفكرة العامة لـ Factory باعتبارها المكون المسؤول عن اتخاذ قرار التوجيه واختيار المسار المناسب لتنفيذ الطلب، دون الدخول في تفاصيل تنفيذ نمط التصميم أو مخططاته البرمجية (محتوى السلايدات 99-100)، لأن هذه التفاصيل ستدرس لاحقاً ضمن مقرر متخصص في أنماط التصميم.



"There's no shame in fear.

What matters is how we face it."

المناقلات التصريحية (Declarative Transactions)

تُعد المناقلات (Transactions) من أهم الآليات المستخدمة للحفاظ على صحة البيانات واتساقها أثناء تنفيذ العمليات، وخاصة في الأنظمة الموزعة التي تتعامل مع أكثر من خدمة أو أكثر من قاعدة بيانات.

ملاحظة إمتحانية:

المطلوب هو فهم طريقة عمل كل نوع من أنواعها، ومعرفة القرار الذي تتخذه طبقة Facade عند استقبال الطلب.

دور ال Facade في إدارة المناقلة:

عند وصول طلب إلى النظام، تقوم ال Facade أولاً بفحص حالة المناقلة الحالية (Transaction Context)، ثم تقرأ نوع السياسة المطلوبة (Transaction Propagation) المصرح بها، وبعد ذلك تتخذ القرار المناسب. وبناءً على هذه المعلومات قد تقوم بأحد الإجراءات التالية:

1. الانضمام إلى مناقلة موجودة.
2. إنشاء مناقلة جديدة.
3. تنفيذ العملية دون مناقلة.
4. إظهار استثناء (Exception) إذا كانت شروط التنفيذ غير متحققة.

وبذلك تكون ال Facade هي الجهة المسؤولة عن اتخاذ القرار المناسب قبل استدعاء الخدمة المطلوبة.

أنواع المناقلات التصريحية:

1. Required:

يُعد هذا النوع الأكثر استخداماً.

- إذا كانت هناك مناقلة مفتوحة بالفعل، فإن العملية تنضم إليها وتستمر ضمنها.
- أما إذا لم تكن هناك مناقلة، فإن النظام ينشئ مناقلة جديدة قبل تنفيذ العملية.

أي أن هذا النوع يضمن دائماً تنفيذ العملية داخل مناقلة.

2. Required New (Requires New):

في هذا النوع يتم دائماً إنشاء مناقلة جديدة مستقلة، حتى لو كانت هناك مناقلة مفتوحة مسبقاً.

وبذلك تُنفذ العملية داخل مناقلة منفصلة تماماً عن أي مناقلة أخرى.

3. Mandatory:

يشترط هذا النوع وجود مناقلة مفتوحة مسبقاً.

فإذا وصلت العملية ولم تكن هناك مناقلة نشطة، فإن النظام لا ينشئ واحدة جديدة، وإنما يُظهر مباشرة استثناء (Exception) لأن شرط التنفيذ غير محقق.

4. Supported:

- إذا كانت هناك مناقلة مفتوحة، فإن العملية تنضم إليها.
- أما إذا لم توجد مناقلة، فإن العملية تُنفذ بصورة طبيعية دون إنشاء مناقلة جديدة.

وبالتالي فإن وجود المناقلة في هذا النوع يعد اختياريًا وليس إلزاميًا.

مقارنة مختصرة:

النوع	عند وجود مناقلة	عند عدم وجود مناقلة
Required	ينضم إليها	ينشئ مناقلة جديدة
Required New	ينشئ مناقلة جديدة مستقلة	ينشئ مناقلة جديدة
Mandatory	ينضم إليها	يظهر Exception
Supported	ينضم إليها	ينفذ بدون مناقلة

فائدة اريبيزية:

أكد الدكتور أن أسئلة الامتحان تركز غالباً على إعطاء حالة معينة ثم السؤال:

- ماذا سيفعل النظام؟

وللإجابة يجب تحديد:

1. هل توجد مناقلة مفتوحة أم لا؟
2. ما هو نوع الـ Transaction المطلوب (Required أو Required New أو Mandatory أو Supported)؟
3. ما القرار الذي ستتخذه الـ Facade؟

وبناءً على ذلك يتم تحديد فيما إذا كانت العملية ستندمج إلى مناقلة موجودة، أو ستنشئ مناقلة جديدة، أو ستعمل بدون مناقلة، أو ستظهر استثناء.

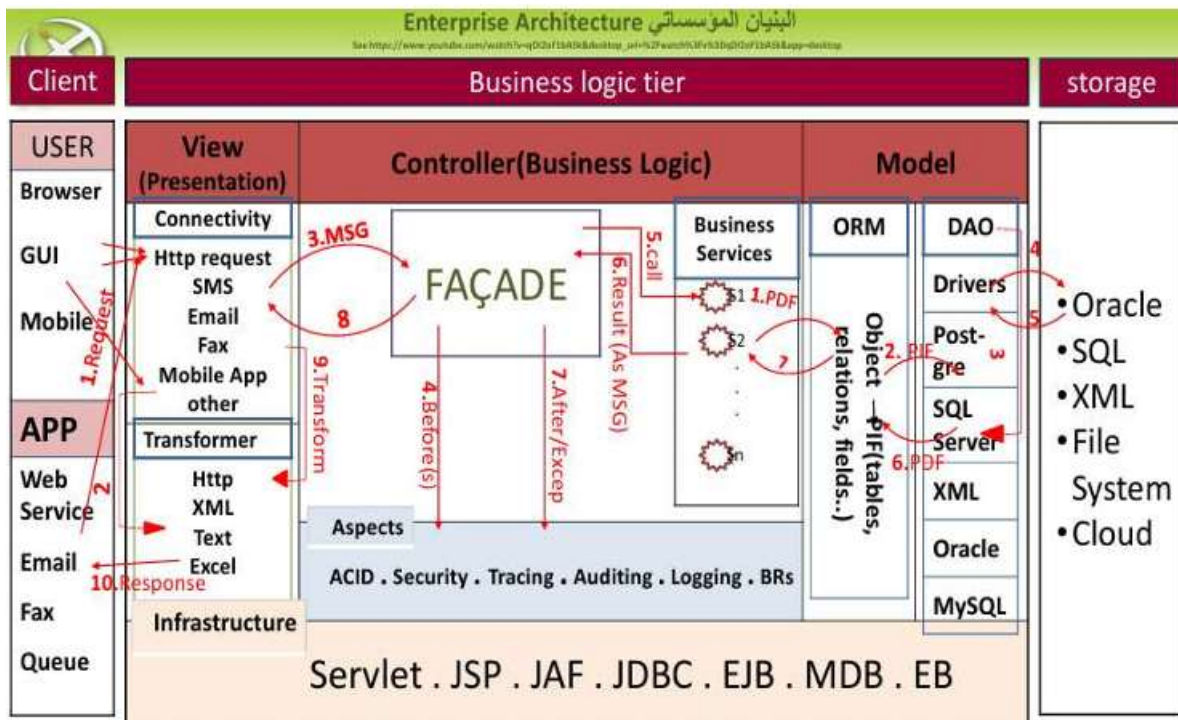
البرمجيات الوسيطة (Middleware)

■ بعد بناء البنية المعمارية (Architecture) وتقسيم النظام إلى طبقات، واستخدام أنماط التصميم (Design Patterns) لفصل المسؤوليات بين مكونات النظام، نصل إلى طبقة أعلى مسؤولة عن ربط هذه المكونات ببعضها، وهي طبقة البرمجيات الوسيطة (Middleware)، والتي تُعرف في الأنظمة الكائنية الموزعة أيضاً باسم وسيط طلبات الأغراض (- Object Request Broker (ORB).

■ جاء ظهور البرمجيات الوسيطة استجابةً للحاجة إلى تحقيق المتطلبات غير الوظيفية (Non-Functional Requirements)، مثل الوثوقية، وقابلية التوسع، وتوزيع الأحمال، والتكامل بين الأنظمة المختلفة. فبعد أن أصبحت الأنظمة الحديثة تتكون من عدد كبير من المكونات والخدمات المستقلة، لم يعد الربط المباشر بينها عملياً، بل أصبح من الضروري وجود طبقة وسيطة تتولى إدارة عملية التواصل بينها.

ملاحظة توضيحية:

إذا نظرنا إلى المخطط المعماري التالي، نجد أن الأسهم الحمراء تعبر برمجياً عن الاستدعاءات وتبادل الرسائل بين الطبقات مثل استدعاء ال Facade للخدمات وال Aspects وصولاً إلى ال DAO هذه الأسهم الحمراء والاتصالات المتداخلة هي برمجياً وجوهرياً ما تمثله وتديره طبقة ال Middleware لتضمن ترابط النظام وتكامله.



أين تقع البرمجيات الوسيطة؟

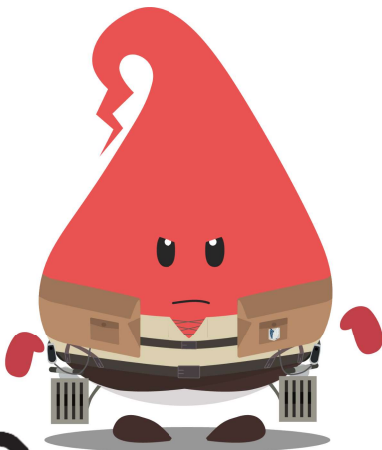
يقع ال Middleware برمجياً فوق نظام التشغيل (OS) والشبكة، وتحت طبقة التطبيقات (Application Layer) وهو يعمل كطبقة وسيطة ممتدة وموزعة تربط المكونات والعمليات المختلفة وتدير التواصل بينها، وليس مجرد حاجز بين ال Request وال Response للمستخدم النهائي فقط.

العمليات (Processes) والخيطوط (Threads)

- في الأنظمة الحديثة لا يتكون التطبيق عادةً من برنامج واحد، وإنما من مجموعة من البرامج أو الخدمات التي تتعاون فيما بينها لإنجاز العمل. ويسمى كل برنامج عملية (Process)، بينما تتكون كل عملية من خيط تنفيذ واحد أو أكثر (Threads).
- تشترك الخيطوط التابعة للعملية نفسها في الذاكرة المشتركة (Heap) بينما يمتلك كل خيط مكدهاً (Stack) خاصاً به، لذلك تستطيع الخيطوط تبادل البيانات فيما بينها بسهولة وسرعة.
- أما العمليات (Processes)، فلكل منها مساحة ذاكرة مستقلة تماماً، ولذلك لا تستطيع التخاطب مباشرة أو مشاركة البيانات كما يحدث بين الخيطوط. وهنا تظهر الحاجة إلى البرمجيات الوسيطة (Middleware)، التي تعمل كوسيط يسمح للعمليات المختلفة بتبادل الرسائل والبيانات فيما بينها، سواء كانت موجودة على الجهاز نفسه أو على أجهزة مختلفة ضمن الشبكة.
- ومن الأمثلة الشائعة على ذلك تطبيقات الويب، حيث لا يتواصل المستخدم مباشرة مع قاعدة البيانات أو الخدمات الداخلية، وإنما يتم التواصل من خلال طبقة وسيطة تستقبل الطلبات، ثم توجهها إلى الخدمة المناسبة، وتعيد النتائج إلى المستخدم.

دور البرمجيات الوسيطة (Middleware)

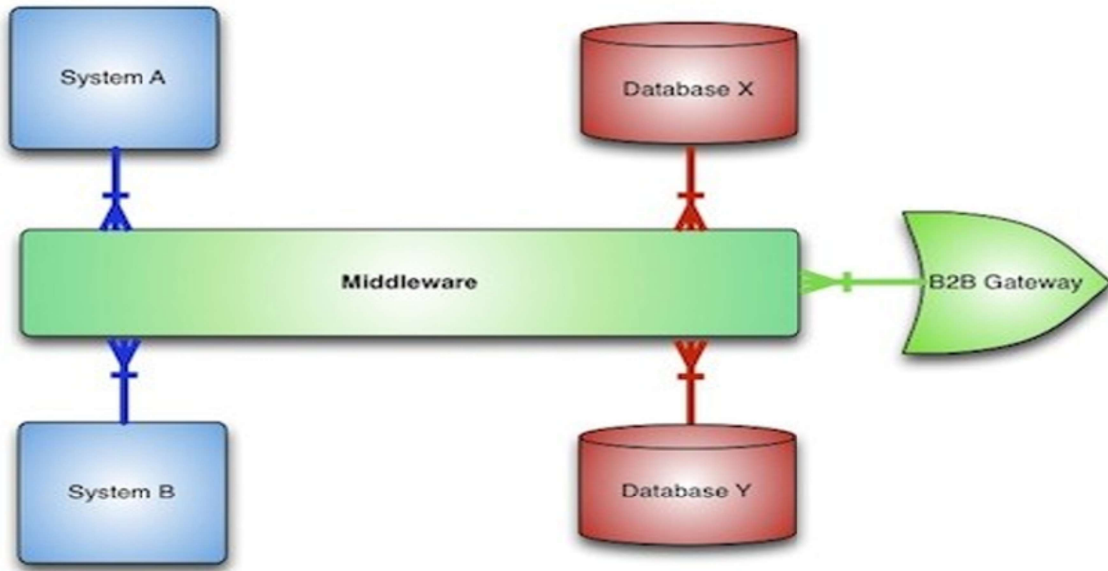
- تُوصف البرمجيات الوسيطة بأنها Broker (وسيط)، لأنها لا تنفذ الخدمة بنفسها، وإنما تستقبل الرسائل أو الأغراض (Objects)، ثم تحدد الجهة المناسبة التي يجب أن ترسل إليها، وبعد تنفيذ الخدمة تعيد النتيجة إلى التطبيق الذي أرسل الطلب.
- وفي الأنظمة الموزعة لا يمكن استخدام الاستدعاء المباشر للتوابع (Method Calls) كما يحدث داخل البرنامج الواحد، لأن كل عملية تمتلك مساحة ذاكرة مستقلة. لذلك تعتمد البرمجيات الوسيطة على نقل الرسائل أو الأغراض بين العمليات المختلفة، ثم تقوم الجهة المستقبلة بتنفيذ العملية المطلوبة وإعادة النتيجة.
- كما ساهم تطور البرمجيات الوسيطة في ظهور العديد من التقنيات الحديثة، مثل خدمات الويب (Web Services)، والتكامل بين التطبيقات المختلفة، وتشغيل البرامج الموزعة عبر الشبكات، بحيث أصبحت التطبيقات تتواصل فيما بينها بطريقة موحدة بغض النظر عن لغة البرمجة أو نظام التشغيل المستخدم.



"Tataikai, Tataikai, TATAKAI!!"

أهمية البرمجيات الوسيطة

- عند ازدياد عدد البرامج داخل النظام، يصبح الربط المباشر بينها معقداً للغاية، إذ يؤدي إنشاء قناة اتصال مستقلة بين كل برنامج وآخر إلى تضخم عدد الاتصالات وتعقيد النظام بصورة كبيرة.
- تعالج البرمجيات الوسيطة هذه المشكلة من خلال توفير لغة وبروتوكول اتصال موحد؛ إذ تتواصل جميع التطبيقات مع الـ Middleware فقط، بينما تتولى هي عملية توجيه الرسائل إلى الوجهة الصحيحة، مما يقلل الترابط المباشر بين التطبيقات، ويسهل تطوير النظام وصيانته والعمل الجماعي عليه.
- ويبين الشكل التالي الدور الذي تقوم به البرمجيات الوسيطة في ربط الأنظمة المختلفة وقواعد البيانات ضمن منصة اتصال واحدة.



الخصائص الأساسية للبرمجيات الوسيطة

لكي تستطيع البرمجيات الوسيطة العمل بكفاءة في الأنظمة الموزعة، يجب أن تحقق عدداً من الخصائص الأساسية، أهمها:

1. الاستقلال عن منصة العمل (Platform Independence)

يجب أن تكون مستقلة عن نظام التشغيل، ولغة البرمجة، والعتاد المستخدم، بحيث تتمكن التطبيقات المختلفة من التواصل فيما بينها مهما اختلفت بيئات تنفيذها.

2. الوثوقية (Reliability)

من أهم الخصائص هي ضمان وصول الرسالة **مرة واحدة ومرة واحدة فقط (Exactly Once)**.

وتظهر أهمية هذه الخاصية في التطبيقات الحساسة مثل الأنظمة البنكية. فلو طلب مستخدم سحب مبلغ من الصراف الآلي (ATM)، ثم حدث انقطاع أثناء التنفيذ، فلا يجوز أن تنفذ عملية السحب مرتين، كما لا يجوز أن تضيع العملية بعد خروجها من التطبيق المرسل. ولذلك تستمر البرمجيات الوسيطة بمحاولة إيصال الرسالة حتى يتم تنفيذها بصورة صحيحة، مع ضمان عدم تنفيذها أكثر من مرة.

3. دعم المناقلات (Transactions)

ينبغي أن تدعم البرمجيات الوسيطة تنفيذ المناقلات الموزعة، مثل بروتوكول Two-Phase Commit (2PC)، لضمان نجاح جميع أجزاء العملية أو فشلها بالكامل دون الوصول إلى حالة غير متناسقة.

دعم المناقلات الموزعة (Distributed Transactions):

ليست جميع البرمجيات الوسيطة قادرة على إدارة المناقلات الموزعة، أي العمليات التي تُنفذ على أكثر من تطبيق أو أكثر من قاعدة بيانات في الوقت نفسه.

فعند تنفيذ عملية موزعة، قد ينجح جزء منها بينما يفشل جزء آخر، وفي هذه الحالة يجب أن يكون النظام قادراً على اتخاذ قرار موحد: إما تثبيت جميع التغييرات (Commit) أو إلغاؤها جميعاً (Rollback). وتُحقق هذه المهمة عادة باستخدام بروتوكول Two-Phase Commit (2PC).

- تُكمن أهمية هذه الآلية في الحفاظ على اتساق البيانات (Consistency) بين جميع الأنظمة المشاركة في العملية. فلو قام تطبيق أول بتحديث قاعدة بياناته، ثم استدعى تطبيقاً آخر ليحدث قاعدة بيانات مختلفة، وبعد ذلك فشلت العملية، فيجب أن تراجع جميع التطبيقات عن التعديلات التي أجرتها، وليس التطبيق الأول فقط.
- أما إذا كانت البرمجيات الوسيطة لا تدعم Two-Phase Commit، فإن كل تطبيق يدير مناقلته بصورة مستقلة، مما قد يؤدي إلى تراجع أحد التطبيقات عن تعديلاته، بينما تبقى تعديلات التطبيقات الأخرى كما هي، فتفقد البيانات اتساقها.

تقنيات لا تدعم Two-Phase Commit:

في المحاضرة تمت الإشارة إلى أن عدداً من التقنيات لا يوفر دعماً مباشراً لهذه الخاصية، ومن أهمها:

- Sockets
- HTTP
- TCP/IP
- RPC (Remote Procedure Call)
- Java RMI بصيغتها القياسية.

أما تقنيات مثل CORBA و EJB فتوفر دعماً للمناقلات الموزعة، ولذلك تُستخدم في الأنظمة المؤسسية الكبيرة.

4. دعم التخاطب المتزامن وغير المتزامن

قد يحتاج التطبيق أحياناً إلى انتظار نتيجة الطلب مباشرة (Synchronous Communication)، بينما قد يرغب في حالات أخرى بإرسال الطلب ومتابعة تنفيذ أعمال أخرى دون انتظار الاستجابة (Asynchronous Communication) كما يحدث عند إرسال البريد الإلكتروني أو الإشعارات.

5. الاستدعاء بالاسم (Location Transparency)

يمكن للتطبيق طلب خدمة باستخدام اسمها فقط دون الحاجة لمعرفة عنوان المخدم (IP Address) أو مكان وجوده الفعلي، بينما تتولى البرمجيات الوسيطة تحديد مكان تنفيذ الخدمة المناسب، مما يسهل توزيع الأحمال (Load Balancing)، ويتيح نقل الخدمات بين المخدمات دون الحاجة إلى تعديل برامج العملاء.

فعندما تكتب google.com استدعاء بالاسم، تقوم الأنظمة الوسيطة بفحص السيرفرات. إذا كان السيرفر واقعاً، يقوم النظام فوراً وبشكل مخفي بتوجيه طلبك إلى السيرفر البديل المتاح. أما لو كنت تستدعي سيرفر معين بـ IP محدد، ستفشل العملية مباشرة ولن تفتح الصفحة، لأنك أجبرت النظام على الذهاب للجهاز واحد بعينه وهو متوقف حالياً.

6. التسلسل (Serialization)

تستطيع البرمجيات الوسيطة تحويل الأغراض والبيانات المعقدة إلى صيغة قابلة للإرسال عبر الشبكة، ثم إعادة بنائها عند الطرف المستقبل، وتعد JSON وXML من أشهر الصيغ المستخدمة لهذا الغرض.

7. التوسع (Scalability)

ينبغي أن تكون البرمجيات الوسيطة قادرة على التكيف مع ازدياد حجم الطلبات (Traffic) وعدد المستخدمين دون الحاجة إلى إعادة تصميم النظام.

8. دعم أنماط/بنى الاتصال المختلفة

مثل الاتصال نقطة إلى نقطة (Point-to-Point)، والإذاعة (Broadcast)، إضافة إلى العمل عبر بروتوكولات وتقنيات اتصال متعددة مثل Sockets و HTTP والشبكات اللاسلكية وغيرها.

ملاحظة:

ليس من الضروري أن تدعم جميع البرمجيات الوسيطة جميع الخصائص السابقة، وإنما تختلف إمكانياتها باختلاف الغرض الذي صممت من أجله. وسنلاحظ هذا في فقرة أنواع البرمجيات الوسيطة لاحقاً.

الحاجة إلى التسلسل (Serialization):

عند تبادل البيانات بين مكونات النظام، تختلف طريقة نقل الكائنات (Objects) بحسب مكان وجود الطرف المستقبل.

1. إذا كان التخاطب يتم داخل العملية (Process) نفسها:
فلا توجد حاجة لنسخ الكائن، لأن جميع الخيوط (Threads) تشترك في الذاكرة نفسها (Heap). لذلك يكفي تمرير عنوان الكائن (Reference)، فيتمكن الطرف الآخر من الوصول إلى الكائن مباشرة دون إنشاء نسخة جديدة.
2. أما إذا كان التخاطب يتم بين عمليتين مختلفتين (Processes)، أو بين جهازين مختلفين عبر الشبكة:
فلا يمكن إرسال عنوان الذاكرة، لأن لكل عملية مساحة ذاكرة مستقلة، ولا يكون عنوان الكائن صالحاً خارج العملية التي أنشئ فيها، لذلك يجب أولاً نسخ الكائن وتحويله إلى شكل تسلسلي (Serialization) يمكن إرساله عبر الشبكة، ثم يُعاد بناؤه عند الطرف المستقبل من خلال عملية Deserialization.

ملاحظة (سؤال امتحاني):

حتى يمكن إرسال الرسالة (Message) بين عمليتين مختلفتين، يجب أن تكون قابلة للتسلسل (Serializable)

JSON وعلاقتها بالتسلسل (Serialization)

بعد تحويل الكائن إلى شكل تسلسلي، يحتاج النظام إلى صيغة قياسية لتمثيله وإرساله، وتُعد JSON (JavaScript Object Notation) من أكثر الصيغ استخداماً لهذا الغرض، إذ تمثل الشكل النصي المتسلسل (Serial Form) للكائنات الموجودة في الذاكرة.

سؤال امتحاني:

■ ما هي JSON؟

■ الإجابة: هي التمثيل النصي المتسلسل (Serial Form) للأغراض (Objects) الموجودة في الذاكرة.

تظهر الحاجة إلى JSON عند الرغبة في نقل كائن من عملية (Process) إلى أخرى أو بين جهازين عبر الشبكة، إذ لا يمكن إرسال الكائن كما هو داخل الذاكرة، وإنما يجب أولاً تحويله إلى صيغة نصية قابلة للإرسال والتخزين، وهي العملية التي تُعرف باسم Serialization. وعند وصول البيانات إلى الطرف الآخر، تُعاد إلى شكلها الأصلي من خلال عملية Deserialization.

وقد أصبحت JSON المعيار الأكثر انتشاراً لهذا الغرض لأنها تمتاز بما يلي:

1. أصغر حجماً من XML.
 2. سهولة القراءة والفهم بالنسبة للمبرمج.
 3. مدعومة في معظم لغات البرمجة الحديثة مثل JavaScript وPython وJava وغيرها.
 4. مناسبة لنقل البيانات بين الأنظمة المختلفة بغض النظر عن لغة البرمجة أو منصة التشغيل.
- ورغم أن تمثيل الكائن داخل الذاكرة يكون أكثر كفاءة من ناحية الحجم (فقد يُخزن العدد الصحيح مثلاً في أربعة بايت فقط)، فإن تمثيله بصيغة JSON يحتاج إلى مساحة أكبر لأنه يُخزن على شكل أحرف نصية. ومع ذلك، فإن هذه الزيادة في الحجم تُعد ثمناً مقبولاً مقابل سهولة القراءة، وإمكانية التخزين، وسهولة نقل البيانات بين التطبيقات المختلفة.

لهذا السبب أصبحت JSON الصيغة القياسية المستخدمة في معظم خدمات الويب (Web Services) وواجهات البرمجة (APIs) لتبادل البيانات بين الأنظمة المختلفة.

وسيط طلبات الأغراض (Object Request Broker - ORB)

يُطلق على البرمجيات الوسيطة المخصصة لتبادل الأغراض (Objects) في الأنظمة الموزعة اسم **وسيط طلبات الأغراض (Object Request Broker - ORB)**، ويُعرف أيضاً باسم Object Middleware.

- تتمثل مهمة الـ ORB في العمل كوسيط (Broker) بين التطبيقات أو الأغراض الموزعة، بحيث يسمح لكائن موجود في برنامج أو جهاز معين باستدعاء طرائق (Methods) لكائن آخر موجود في برنامج أو جهاز مختلف، وكأنهما موجودان ضمن العملية نفسها.

- ويتولى الـ ORB جميع التفاصيل المتعلقة بإرسال الطلبات عبر الشبكة، واستقبال النتائج، ونقل الأغراض بين الأطراف المختلفة، مما يخفي عن المبرمج تعقيدات الاتصال والتخاطب بين الأنظمة، ويوفر طبقة تجريد تسمح للتطبيقات بالتواصل دون الحاجة لمعرفة مكان وجود الكائنات أو كيفية الوصول إليها.
- ويُعد الـ ORB أحد المكونات الأساسية في البنية الكائنية الموزعة (Object-Oriented Distributed Architecture)، ويشكل الأساس الذي بُنيت عليه العديد من تقنيات البرمجيات الوسيطة الحديثة.

ملاحظة (سؤال امتحاني):

ينبغي معرفة معنى الاختصار ORB، وهو:

Object Request Broker

ويشير إلى البرنامج الوسيط المسؤول عن إدارة التخاطب بين الأغراض في الأنظمة الموزعة

أساليب التكامل بين الأنظمة (ETL و EAI)

بصرف النظر عن نوع البرمجيات الوسيطة (Middleware) المستخدمة، توجد فلسفتان أساسيتان لتحقيق التكامل بين الأنظمة وتبادل البيانات، وهما ETL و EAI، ويختلف كل منهما في طريقة نقل البيانات والغرض من استخدامها.

أولاً: ETL (Extract - Transform - Load):

- يرمز ETL إلى استخراج البيانات (Extract)، ثم تحويلها (Transform) إلى الصيغة المناسبة، وأخيراً تحميلها (Load) إلى نظام أو قاعدة بيانات أخرى.
- يعتمد هذا الأسلوب على تنفيذ عدد قليل من المناقلات (Transactions)، إلا أن كل مناقلة تنقل كمية كبيرة جداً من البيانات دفعة واحدة، لذلك يستخدم غالباً في مستودعات البيانات (Data Warehouses) وأنظمة ذكاء الأعمال (Business Intelligence - BI)، حيث يكون الهدف هو تحليل البيانات التاريخية واتخاذ القرارات الاستراتيجية، وليس الحصول على معلومات لحظية.
- فعلى سبيل المثال، قد يقوم النظام ب جلب بيانات جميع الموظفين من نظام الموارد البشرية، وبيانات جميع الطلاب من نظام شؤون الطلاب، ثم دمجها وتحليلها لتحديد أفضل مسارات حافلات الجامعة. كما يمكن تحميل بيانات أسعار الأسهم أو العملات ليوم كامل بهدف إجراء التحليلات والإحصاءات لاحقاً.
- ويستخدم هذا الأسلوب عادةً الباحثون والمحللون الذين يهتمون بتحليل كميات كبيرة من البيانات، ولا يشترط أن تكون المعلومات محدثة في اللحظة نفسها.



"We're in the endgame now."

ثانياً: EAI (Enterprise Application Integration)

- يرمز EAI إلى تكامل تطبيقات المؤسسة، ويهدف إلى تحقيق التكامل اللحظي بين التطبيقات المختلفة أثناء تشغيل النظام.
- يعتمد هذا الأسلوب على تنفيذ عدد كبير جداً من المناقلات الصغيرة، بحيث تحتوي كل عملية على كمية محدودة من البيانات، ولكنها تتم بصورة مستمرة وفورية (Real-Time).
- ويستخدم بشكل واسع في تطبيقات الأعمال الإلكترونية (e-Business)، حيث تتطلب العمليات الحصول على معلومات محدثة في اللحظة نفسها. فعلى سبيل المثال، عند شراء منتج من متجر إلكتروني، يجب على النظام التأكد مباشرة من توفر المنتج في المستودع، ثم تحديث الكمية بعد إتمام عملية البيع، وقد يحتاج أيضاً إلى الاستعلام فوراً عن توفر المنتج لدى المورد إذا لم يكن متوفراً محلياً.
- كما يستخدم في الأنظمة المالية وأسواق الأسهم التي تعتمد على البيانات اللحظية (Tick Data)، حيث تتغير الأسعار باستمرار ويجب أن تكون المعلومات متاحة خلال أجزاء من الثانية.

مقارنة بين EAI و ETL:

يمكن تلخيص الفرق الأساسي بين الأسلوبين كما يلي:

- ETL: عدد قليل من المناقلات، لكن كل منها ينقل كمية كبيرة من البيانات، ويستخدم للتحليل واتخاذ القرارات الاستراتيجية.
- EAI: عدد كبير جداً من المناقلات، لكن كل منها ينقل كمية صغيرة من البيانات، ويستخدم في التطبيقات التشغيلية التي تتطلب معلومات لحظية.

وتمت الإشارة في المحاضرة إلى أن EAI أكثر تكلفة من ETL، لأنه يحتاج إلى مخدمات عالية الأداء وموارد كبيرة قادرة على معالجة آلاف الطلبات في الزمن الحقيقي، في حين أن ETL يعمل عادةً على دفعات (Batch Processing)، مما يجعل متطلباته التشغيلية أقل.

جدول المقارنة الشامل:

وجه المقارنة	EAI	ETL
Transactions	few	many
Amount of data	enormous	small
Integration	data	applications
Purpose	operations & strategic	operations
Business care	Business intelligence	IT, E-business

	Decision making	Better workflow
	One view of customer	Data entry once
Common	transformations	
	Meta data	
	connectors	
	Service oriented architectures	

ملاحظات:

- تمت الإشارة إلى أن النظام الواحد قد يستخدم الأسلوبين معاً بحسب طبيعة البيانات؛ فقد تجلب أسعار العملات الرسمية مرة واحدة يومياً باستخدام ETL لأنها لا تتغير باستمرار، بينما تتابع أسعار الأسهم أو بيانات التداول اللحظية باستخدام EAI نظراً لتغيرها المستمر.
- كلا الأسلوبين يشتركان في الحاجة إلى تحويل البيانات (Transformation)، ووجود بيانات وصفية (Metadata)، إضافة إلى Connectors أو وسائل ربط تسمح بالتواصل مع الأنظمة المختلفة، بينما يكمن الاختلاف الأساسي في طريقة نقل البيانات وتوقيت استخدامها وليس في نوع البرمجيات الوسيطة المستخدمة.